

Nombre: _____ Legajo: _____

Pregunta	1	2	3	Total
Puntos	3	4	3	10
Calificación				

INDICACIONES:

1. La duración del examen es de 3 horas.
2. Las respuestas al examen se deben presentar de manera prolija, escritas como si fueran parte del informe de un trabajo práctico, enviadas en un archivo PDF. Dicho archivo debe contener, en un apéndice, todo el código de Python.
3. Se deben enviar todos los archivos de Python utilizados para la resolución.
4. Se puede utilizar cualquier recurso impreso, en formato digital o en Internet que haya sido producido antes de la fecha del examen. Lo correcto es citar adecuadamente los recursos utilizados, salvo que los mismos se correspondan con los apuntes de la materia.
5. Como deben enviar varios archivos y el sistema sólo permitirá enviar uno, se recomienda comprimirllos y enviarlos como parte de un archivo ZIP.
6. Está prohibido recibir ayuda de otros alumnos de esta materia o de cualquier otra persona, de manera directa o indirecta (salvo que sea a través de un recurso mencionado en el ítem anterior).
7. Les pedimos que estén con la cámara encendida, apuntando hacia ustedes, en una reunión de Google Meet cuyo enlace es:
<https://meet.google.com/diz-jnye-ibu>
8. Consultas sobre el examen deben hacerse por ese medio. Les pedimos que, en la medida de lo posible, alcen la mano y esperen a ser atendidos. Podrán hacer las preguntas abriendo el micrófono o por el chat.
9. Dado que serán enviados varios archivos, no podremos verificar inmediatamente si la entrega está completa. Por ello, les pedimos que no consulten sobre la completitud de la entrega apenas terminado el examen. Les sugerimos que se tomen tiempo para chequearlo ustedes antes de enviarlo o antes del fin del tiempo estipulado para el examen.
10. Insistimos sobre un punto: no se copien, utilicen su propio código.
11. Buena suerte.

1. (3 ptos.) Sistemas lineales iterativos

Si su legajo es par:

- (a) Escriba una función que implemente el método de Jacobi para resolver el sistema de ecuaciones lineales $\mathbf{A}\vec{x} = \vec{b}$. La función debe ser `jacobi(A,b,tol)` donde $\mathbf{A}$ se corresponde con la matriz $\mathbf{A}$, $\mathbf{b}$ con el vector $\vec{b}$ y `tol` es la máxima norma infinito de la diferencia entre los vectores en dos aproximaciones consecutivas. La función debe devolver la aproximación final y el número de iteraciones, en ese orden.

- (b) Sea `legajo` su número de legajo. Ejecute el siguiente código:

```
import numpy as np

rng = np.random.default_rng(legajo)

M = 5

A = rng.random((M,M))

b = rng.random((M))

for k in range(M):
    A[k,k] = ((-1)**k)*(np.sum(A[k,:])+k)

x1,it1 = jacobi(A,b,legajo*1e-14)
```

- Explique el por qué converge el algoritmo con esta matriz.
- Dé el resultado del algoritmo y el número de iteraciones.

Si su legajo es impar:

- (a) Escriba una función que implemente el método de Gauss-Seidel para resolver el sistema de ecuaciones lineales $\mathbf{A}\vec{x} = \vec{b}$. La función debe ser `gaussseidel(A,b,tol)` donde $\mathbf{A}$ se corresponde con la matriz $\mathbf{A}$, $\mathbf{b}$ con el vector $\vec{b}$ y `tol` es la máxima norma infinito de la diferencia entre los vectores en dos aproximaciones consecutivas. La función debe devolver la aproximación final y el número de iteraciones, en ese orden.

- (b) Sea `legajo` su número de legajo. Ejecute el siguiente código:

```
import numpy as np

rng = np.random.default_rng(legajo)

M = 5

A = rng.random((M,M))

b = rng.random((M))

for k in range(M):
    A[k,k] = ((+1)**k)*(np.sum(A[k,:])+k)

x1,it1 = gaussseidel(A,b,legajo*1e-14)
```

- Explique el por qué converge el algoritmo con esta matriz.
- Dé el resultado del algoritmo y el número de iteraciones.

2. (4 ptos.) Ecuaciones diferenciales ordinarias

- (a) Escriba una función implemente el método de Taylor de orden 3 para resolver numéricamente el problema de valor inicial con ecuaciones diferenciales ordinarias. La función debe ser `taylor3(dx,ddx,dddx,x0,t0,tf,n)` donde

- `dx` es $d\vec{x}/dt$;
- `ddx` es $d^2\vec{x}/dt^2$;
- `dddx` es $d^3\vec{x}/dt^3$;
- `x0` es $\vec{x}(t_0) = \vec{x}_0$;
- `t0` es t_0 , el tiempo inicial;
- `tf` es t_f , el tiempo final de simulación;
- `n` es el número de puntos entre `t0` y `tf`, incluyéndolos.

Las funciones `dx`, `ddx` y `dddx` deben recibir primero $\vec{x}$ y luego t .

La función `taylor3()` debe devolver los valores $\vec{x}$ en un arreglo Numpy (una fila por instante de tiempo) y el vector de tiempos, en ese orden. `x0` será un vector de columna (con segunda dimensión igual a 1).

La ecuación a resolver es del tipo:

$$\frac{d\vec{x}}{dt} = f(\vec{x}, t) \quad t > t_0,$$

$$\vec{x}(t_0) = \vec{x}_0.$$

- (b) Verifique el correcto funcionamiento de función con un ejemplo.
- (c) Presente gráficos adecuados que muestren la solución de los problemas que se presentan a continuación. El máximo error en módulo en todo el intervalo debe ser menor que 10^{-6} . Explique cómo lo garantiza.

Ayuda: Una ecuación de orden superior puede escribirse como un sistema de ecuaciones diferenciales de primer orden.

Ayuda 2: x_1 y x_2 son la primera y segunda componentes del vector $\vec{x}$, respectivamente.

- Si su número de legajo termina en 0 o 1: resuelva numéricamente para $t \in [0, 2]$

$$\frac{d^2y}{dt^2} = y \cdot (t + 1) \quad t > 0,$$

$$y(0) = \text{número de legajo}/1000,$$

$$\frac{dy(0)}{dt} = 1.$$

- Si su número de legajo termina en 2 o 3: resuelva numéricamente para $t \in [0, 10]$

$$\frac{d\vec{x}}{dt} = \begin{pmatrix} x_2^3 \\ \sin(t) \end{pmatrix} \quad t > 0,$$

$$\vec{x}(0) = \begin{pmatrix} -1 \\ -60000/\text{legajo} \end{pmatrix}.$$

- Si su número de legajo termina en 4 o 5: resuelva numéricamente para $t \in [0, 3]$

$$\frac{d\vec{x}}{dt} = \begin{pmatrix} \sin(x_2) \\ \sin(x_1) \end{pmatrix} \quad t > 0,$$

$$\vec{x}(0) = \begin{pmatrix} +60000/\text{legajo} \\ -60000/\text{legajo} \end{pmatrix}.$$

- Si su número de legajo termina en 6 o 7: resuelva numéricamente para $t \in [0, 10]$

$$\frac{d\vec{x}}{dt} = \begin{pmatrix} \sin(t) \\ e^{x_1} \end{pmatrix} \quad t > 0,$$

$$\vec{x}(0) = \begin{pmatrix} -1 \\ -\text{legajo}/100000 \end{pmatrix}.$$

- Si su número de legajo termina en 8 o 9: resuelva numéricamente para $t \in [-5, 5]$

$$\begin{aligned}\frac{d^2y}{dt^2} &= -y + e^{-t} & t > -5, \\ y(-5) &= \text{número de legajo}/1000, \\ \frac{dy(-5)}{dt} &= 1.\end{aligned}$$

3. (3 ptos.) Cuadrados mínimos

Se desea resolver el problema de cuadrados mínimos

$$\vec{x}_{LS} = \arg \min_{\vec{x}} \left\| \mathbf{A}\vec{x} - \vec{b} \right\|_2.$$

- (a) Escriba un algoritmo para la resolución de este problema para el caso específico en que el número de columnas (linealmente independientes) de $\mathbf{A}$ es 2. Su algoritmo **no** debe tener lazos **for** ni **while**. El algoritmo debe estar en una función llamada `cuadmin(A,b)` donde $\mathbf{A}$ se corresponde con $\mathbf{A}$ y $\mathbf{b}$ con $\vec{b}$. La función debe devolver $\vec{x}_{LS}$ y $\|\mathbf{A}\vec{x}_{LS} - \vec{b}\|_2$ en ese orden.

- Si la suma de los últimos dos dígitos de su legajo termina con 0, 1, 2 o 3, debe resolverlo usando descomposición en valores singulares.
- Si la suma de los últimos dos dígitos de su legajo termina con 4, 5 o 6, debe resolverlo usando descomposición Cholesky.
- Si la suma de los últimos dos dígitos de su legajo termina con 7, 8 o 9, debe resolverlo usando descomposición QR (use Gram-Schmidt).

Nota: no se puede utilizar ninguna rutina de alguna biblioteca de Python para encontrar autovalores, autovectores, valores singulares, descomposición Cholesky, descomposición QR. Sí se pueden usar rutinas para calcular normas de vectores, raíces cuadradas, valores absolutos, etc.

- (b) Probar su programa cuando $\mathbf{A}$ y $\mathbf{b}$ vienen dados por el siguiente código:

```
import numpy as np

rng = np.random.default_rng(legajo)

M = 1000

A = rng.random((M,2))

b = legajo*A[:,0]+legajo/2*A[:,1]+rng.random((M))
```

`legajo` debe ser igual a su número de legajo.

Dar el resultado de los valores de $\vec{x}$ y el error redondeados a 4 decimales.